LAB 4: CNN (AlexNet)

Santosh Mahato Koiri (ACE079BCT059)

0. Background

CNNs are designed for image data. They use local connectivity, weight sharing (convolution), and spatial pooling — making them translation-invariant and far more efficient than plain fully connected networks for vision tasks.

AlexNet (Krizhevsky et al., 2012) won ILSVRC 2012 by a large margin and kick-started modern deep learning. Key innovations: ReLU activations, Dropout, Local Response Normalization, GPU training, and data augmentation.

AlexNet vs LeNet-5 (predecessor):

Feature	LeNet-5 (1998)	AlexNet (2012)
Depth	5 layers	8 layers (5 conv + 3 FC)
Activation	Tanh/Sigmoid	ReLU
Regularization	None	Dropout (p=0.5)
Normalization	None	LRN
Parameters	~60K	~60M
Training	CPU	Multi-GPU

TinyVGG follows VGGNet's philosophy of stacking small 3×3 conv filters in blocks, but with fewer layers for efficiency on small datasets.

CIFAR-10: 60,000 color images (32×32) across 10 classes, split 50K train / 10K test.

```
# Importing Dependencies
import torch
import torch.nn as nn
import torch.optim as optim
import torch.nn.functional as F
import torchvision
import torchvision.transforms as transforms
import numpy as np
import matplotlib.pyplot as plt
import time
from sklearn.metrics import classification_report, confusion_matrix, f1_score
import seaborn as sns
import pandas as pd
```

```
# Check device
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f"Using device: {device}")
print(f"PyTorch version: {torch.__version__}")
```

```
Using device: cuda
PyTorch version: 2.10.0+cu128
```

CUDA is available, so all training runs on the GPU — significantly faster than CPU.

1. Dataset Loading and Preprocessing

We apply **RandomHorizontalFlip** and **RandomCrop** as data augmentation during training to improve generalization. Both train and test sets are normalized using CIFAR-10's per-channel mean and std, which stabilizes gradient descent by centering each feature distribution.

```
CIFAR10_MEAN = (0.4914, 0.4822, 0.4465)
CIFAR10_STD  = (0.2023, 0.1994, 0.2010)

train_transform = transforms.Compose([
    transforms.RandomHorizontalFlip(),
    transforms.RandomCrop(32, padding=4),
    transforms.ToTensor(),
    transforms.Normalize(CIFAR10_MEAN, CIFAR10_STD)
])
test_transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize(CIFAR10_MEAN, CIFAR10_STD)
])

train_dataset = torchvision.datasets.CIFAR10(root='./data', train=True,  downlc
test_dataset  = torchvision.datasets.CIFAR10(root='./data', train=False, downlc

BATCH_SIZE   = 128
train_loader = torch.utils.data.DataLoader(train_dataset, batch_size=BATCH_SIZE
test_loader  = torch.utils.data.DataLoader(test_dataset,  batch_size=BATCH_SIZE

CLASSES = ('airplane', 'automobile', 'bird', 'cat', 'deer',
           'dog', 'frog', 'horse', 'ship', 'truck')

print(f"Training samples : {len(train_dataset)}")
print(f"Test samples     : {len(test_dataset)}")
print(f"Number of classes: {len(CLASSES)}")
print(f"Classes          : {CLASSES}")
print(f"Batch size       : {BATCH_SIZE}")
```

```
print(f"Training batches : {len(train_loader)}")
print(f"Test batches     : {len(test_loader)}")
```

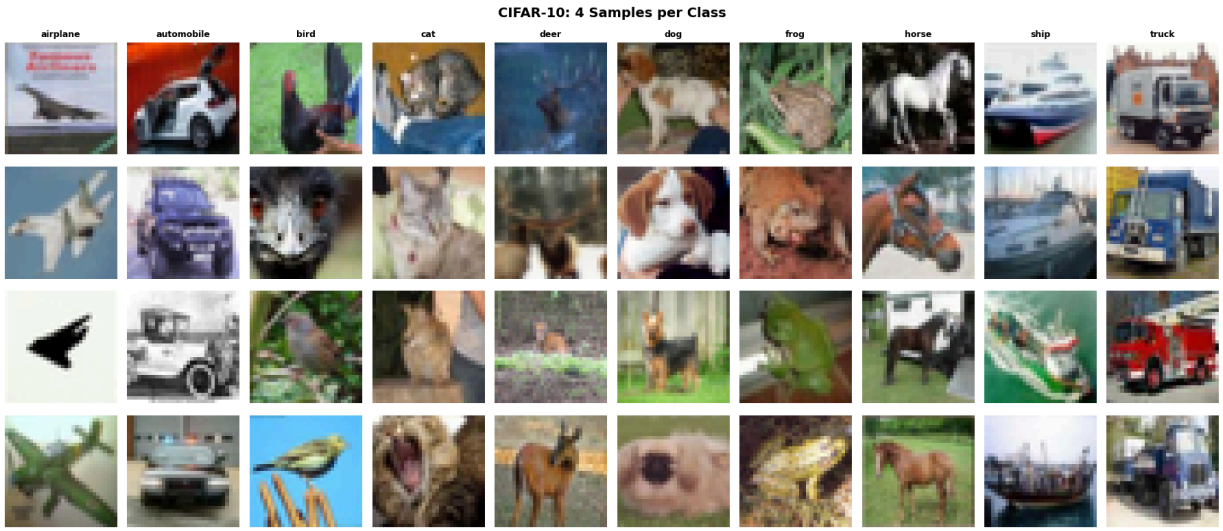
```
100%|██████████| 170M/170M [00:08<00:00, 19.8MB/s]
Training samples : 50000
Test samples     : 10000
Number of classes: 10
Classes          : ('airplane', 'automobile', 'bird', 'cat', 'deer', 'dog', 'fro
Batch size       : 128
Training batches : 391
Test batches     : 79
```

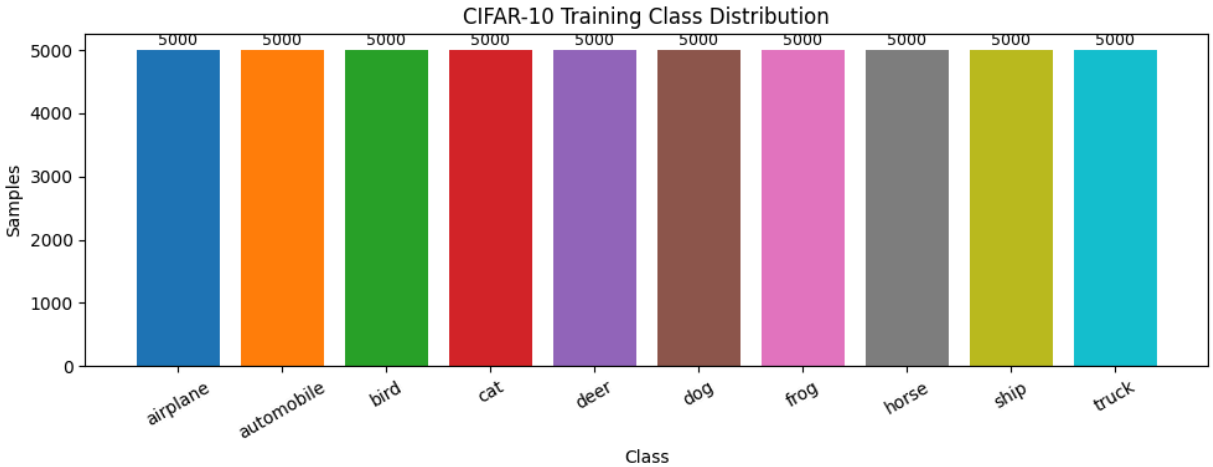
Dataset is perfectly balanced — 5,000 training and 1,000 test images per class — so accuracy is a meaningful metric with no class-imbalance bias.

2. Dataset Visualization

```
raw_test = torchvision.datasets.CIFAR10(root='./data', train=False, download=Fa
                                        transform=transforms.ToTensor())
fig, axes = plt.subplots(4, 10, figsize=(18, 8))
for cls_idx in range(10):
    count = 0
    for img, label in raw_test:
        if label == cls_idx and count < 4:
            axes[count][cls_idx].imshow(img.permute(1, 2, 0).numpy())
            axes[count][cls_idx].axis('off')
            if count == 0:
                axes[count][cls_idx].set_title(CLASSES[cls_idx], fontsize=9, fo
            count += 1
        if count == 4:
            break
plt.suptitle('CIFAR-10: 4 Samples per Class', fontsize=14, fontweight='bold')
plt.tight_layout()
plt.show()

labels_all   = [label for _, label in train_dataset]
class_counts = [labels_all.count(i) for i in range(10)]
plt.figure(figsize=(10, 4))
bars = plt.bar(CLASSES, class_counts, color=plt.cm.tab10.colors)
plt.xlabel('Class'); plt.ylabel('Samples'); plt.title('CIFAR-10 Training Class
plt.xticks(rotation=30)
for bar, cnt in zip(bars, class_counts):
    plt.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 30,
             str(cnt), ha='center', va='bottom', fontsize=9)
plt.tight_layout()
plt.show()
```


CIFAR-10: 4 Samples per Class
airplane
automobile
bird
cat
deer
dog
frog
horse
ship
truck


CIFAR-10 Training Class Distribution
5000
5000
5000
5000
5000
5000
5000
5000
5000
5000
Samples
0
1000
2000
3000
4000
5000
airplane
automobile
bird
cat
deer
dog
frog
horse
ship
truck
Class

Images are only 32×32 pixels — fine details are blurry even to the human eye. Cat vs dog and deer vs horse look visually similar at this resolution, which foreshadows where models will struggle most. Class distribution is perfectly uniform at 5,000 samples each.

3. Model Architectures

3.1 Baseline Neural Network

A fully connected network that takes the flattened image (32×32×3 = 3,072 features) and processes them through two hidden layers. It has no spatial awareness — used as a lower-bound baseline to show how much CNNs improve over plain FC networks.

```
class BaselineNN(nn.Module):
    """Simple FC network — no convolutions, no spatial awareness."""
    def __init__(self):
        super(BaselineNN, self).__init__()
        self.flatten = nn.Flatten()
        self.fc1     = nn.Linear(32 * 32 * 3, 512)
        self.fc2     = nn.Linear(512, 256)
        self.fc3     = nn.Linear(256, 10)
        self.dropout = nn.Dropout(p=0.5)

    def forward(self, x):
        x = self.flatten(x)
        x = F.relu(self.fc1(x))
        x = self.dropout(x)
        x = F.relu(self.fc2(x))
        x = self.dropout(x)
        return self.fc3(x)

def count_parameters(model):
    return sum(p.numel() for p in model.parameters() if p.requires_grad)

baseline_nn = BaselineNN()
print("BaselineNN Architecture:")
print(baseline_nn)
print(f"\nTotal trainable parameters: {count_parameters(baseline_nn):,}")
```

```
BaselineNN Architecture:
BaselineNN(
  (flatten): Flatten(start_dim=1, end_dim=-1)
  (fc1): Linear(in_features=3072, out_features=512, bias=True)
  (fc2): Linear(in_features=512, out_features=256, bias=True)
  (fc3): Linear(in_features=256, out_features=10, bias=True)
  (dropout): Dropout(p=0.5, inplace=False)
)
```

```
Total trainable parameters: 1,707,274
```

1.7M parameters — the majority (~1.57M) are in the first layer alone (3072×512), since flattening loses spatial structure and forces the model to learn everything from raw pixel values.

3.2 AlexNet (Adapted for CIFAR-10)

Original AlexNet was designed for 224×224 images. For CIFAR-10's 32×32 input, we: (1) replace the 11×11 stride-4 first conv with a 3×3 stride-1 conv to avoid collapsing spatial dimensions, (2) remove the first MaxPool, and (3) use AdaptiveAvgPool2d before the FC layers.

```
class AlexNetCIFAR(nn.Module):
    """AlexNet adapted for CIFAR-10 (32x32 input)."""
    def __init__(self, num_classes=10, dropout=0.5):
        super(AlexNetCIFAR, self).__init__()
        self.features = nn.Sequential(
            nn.Conv2d(3,   64,  kernel_size=3, stride=1, padding=1),
            nn.ReLU(inplace=True),
            nn.LocalResponseNorm(size=5, alpha=0.0001, beta=0.75, k=2),
            nn.Conv2d(64,  192, kernel_size=5, stride=1, padding=2),
            nn.ReLU(inplace=True),
            nn.LocalResponseNorm(size=5, alpha=0.0001, beta=0.75, k=2),
            nn.MaxPool2d(kernel_size=2, stride=2),
            nn.Conv2d(192, 384, kernel_size=3, stride=1, padding=1),
            nn.ReLU(inplace=True),
            nn.Conv2d(384, 256, kernel_size=3, stride=1, padding=1),
            nn.ReLU(inplace=True),
            nn.Conv2d(256, 256, kernel_size=3, stride=1, padding=1),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=2, stride=2),
        )
        self.avgpool    = nn.AdaptiveAvgPool2d((4, 4))
        self.classifier = nn.Sequential(
            nn.Dropout(p=dropout),
            nn.Linear(256 * 4 * 4, 4096),
            nn.ReLU(inplace=True),
            nn.Dropout(p=dropout),
            nn.Linear(4096, 4096),
            nn.ReLU(inplace=True),
            nn.Linear(4096, num_classes),
        )

    def forward(self, x):
        x = self.features(x)
        x = self.avgpool(x)
```

```
        x = torch.flatten(x, 1)
        return self.classifier(x)

alexnet = AlexNetCIFAR(num_classes=10)
print("AlexNet (CIFAR-10 Adapted) Architecture:")
print(alexnet)
print(f"\nTotal trainable parameters: {count_parameters(alexnet):,}")
```

```
AlexNet (CIFAR-10 Adapted) Architecture:
AlexNetCIFAR(
  (features): Sequential(
    (0): Conv2d(3, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (1): ReLU(inplace=True)
    (2): LocalResponseNorm(5, alpha=0.0001, beta=0.75, k=2)
    (3): Conv2d(64, 192, kernel_size=(5, 5), stride=(1, 1), padding=(2, 2))
    (4): ReLU(inplace=True)
    (5): LocalResponseNorm(5, alpha=0.0001, beta=0.75, k=2)
    (6): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=Fal
    (7): Conv2d(192, 384, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (8): ReLU(inplace=True)
    (9): Conv2d(384, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (10): ReLU(inplace=True)
    (11): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (12): ReLU(inplace=True)
    (13): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=Fa
  )
  (avgpool): AdaptiveAvgPool2d(output_size=(4, 4))
  (classifier): Sequential(
    (0): Dropout(p=0.5, inplace=False)
    (1): Linear(in_features=4096, out_features=4096, bias=True)
    (2): ReLU(inplace=True)
    (3): Dropout(p=0.5, inplace=False)
    (4): Linear(in_features=4096, out_features=4096, bias=True)
    (5): ReLU(inplace=True)
    (6): Linear(in_features=4096, out_features=10, bias=True)
  )
)

Total trainable parameters: 36,051,786
```

36M parameters — the two large FC layers (4096×4096 each, ~33.6M params combined) dominate. This is very large relative to CIFAR-10's 50K training samples, making overfitting a real risk.

3.3 TinyVGG

Two convolutional blocks, each with two 3×3 Conv layers + BatchNorm + ReLU, followed by MaxPool. BatchNorm (not in original VGG) normalizes activations per mini-batch, enabling more stable training. A small FC classifier follows.

```
class TinyVGG(nn.Module):
    """TinyVGG: Simplified VGG-style architecture for CIFAR-10."""
    def __init__(self, num_classes=10, dropout=0.5):
        super(TinyVGG, self).__init__()
        self.block1 = nn.Sequential(
            nn.Conv2d(3,   64,  kernel_size=3, padding=1),
            nn.BatchNorm2d(64),
            nn.ReLU(inplace=True),
            nn.Conv2d(64,  64,  kernel_size=3, padding=1),
            nn.BatchNorm2d(64),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=2, stride=2),
        )
        self.block2 = nn.Sequential(
            nn.Conv2d(64,  128, kernel_size=3, padding=1),
            nn.BatchNorm2d(128),
            nn.ReLU(inplace=True),
            nn.Conv2d(128, 128, kernel_size=3, padding=1),
            nn.BatchNorm2d(128),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=2, stride=2),
        )
        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(128 * 8 * 8, 512),
            nn.ReLU(inplace=True),
            nn.Dropout(p=dropout),
            nn.Linear(512, num_classes),
        )

    def forward(self, x):
        x = self.block1(x)
        x = self.block2(x)
        return self.classifier(x)

tinyvgg = TinyVGG(num_classes=10)
print("TinyVGG Architecture:")
print(tinyvgg)
print(f"\nTotal trainable parameters: {count_parameters(tinyvgg):,}")
```

```
TinyVGG Architecture:
TinyVGG(
  (block1): Sequential(
    (0): Conv2d(3, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_sta
    (2): ReLU(inplace=True)
    (3): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (4): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_sta
    (5): ReLU(inplace=True)
    (6): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=Fal
  )
  (block2): Sequential(
```

```
      (0): Conv2d(64, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
      (1): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_st
      (2): ReLU(inplace=True)
      (3): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
      (4): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_st
      (5): ReLU(inplace=True)
      (6): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=Fal
    )
    (classifier): Sequential(
      (0): Flatten(start_dim=1, end_dim=-1)
      (1): Linear(in_features=8192, out_features=512, bias=True)
      (2): ReLU(inplace=True)
      (3): Dropout(p=0.5, inplace=False)
      (4): Linear(in_features=512, out_features=10, bias=True)
    )
  )

Total trainable parameters: 4,460,874
```

4.4M parameters — 8× fewer than AlexNet. The classifier is tiny (8192→512→10), keeping the parameter count lean. BatchNorm in each conv block replaces the need for LRN.

4. Loss Function and Optimizer

CrossEntropyLoss is the standard for multi-class classification — it combines LogSoftmax and NLLLoss, penalizing confident wrong predictions heavily. **SGD with Momentum** (same optimizer for all three models as required) with weight decay for L2 regularization. **StepLR** drops the learning rate by ×0.1 every 10 epochs, allowing large steps early and fine-tuning later.

```
def get_optimizer_scheduler(model, lr=0.01, momentum=0.9, weight_decay=5e-4, st
    optimizer = optim.SGD(model.parameters(), lr=lr, momentum=momentum, weight_
    scheduler = optim.lr_scheduler.StepLR(optimizer, step_size=step_size, gamma
    return optimizer, scheduler

criterion = nn.CrossEntropyLoss()
print("Loss Function: nn.CrossEntropyLoss")
print("Optimizer: SGD (lr=0.01, momentum=0.9, weight_decay=5e-4)")
print("Scheduler: StepLR (step_size=10, gamma=0.1)")
```

```
Loss Function: nn.CrossEntropyLoss
Optimizer: SGD (lr=0.01, momentum=0.9, weight_decay=5e-4)
Scheduler: StepLR (step_size=10, gamma=0.1)
```

Using the same optimizer and loss for all three models ensures a fair comparison — any performance difference comes purely from architecture, not training setup.

5. Training and Evaluation Functions

```
def train_one_epoch(model, loader, optimizer, criterion, device):
    model.train()
    total_loss, correct, total = 0.0, 0, 0
    for images, labels in loader:
        images, labels = images.to(device), labels.to(device)
        optimizer.zero_grad()
        outputs = model(images)
        loss    = criterion(outputs, labels)
        loss.backward()
        optimizer.step()
        total_loss += loss.item() * images.size(0)
        _, predicted = outputs.max(1)
        correct += predicted.eq(labels).sum().item()
        total   += labels.size(0)
    return total_loss / total, 100.0 * correct / total

def evaluate(model, loader, criterion, device):
    model.eval()
    total_loss, correct, total = 0.0, 0, 0
    with torch.no_grad():
        for images, labels in loader:
            images, labels = images.to(device), labels.to(device)
            outputs = model(images)
            loss    = criterion(outputs, labels)
            total_loss += loss.item() * images.size(0)
            _, predicted = outputs.max(1)
            correct += predicted.eq(labels).sum().item()
            total   += labels.size(0)
    return total_loss / total, 100.0 * correct / total

def train_model(model, train_loader, test_loader, optimizer, scheduler, criteri
    model = model.to(device)
    history = {'train_loss': [], 'train_acc': [], 'test_loss': [], 'test_acc':
    print(f"\n{'='*65}\nTraining {name}\n{'='*65}")
    t0 = time.time()
    for epoch in range(1, epochs + 1):
        tr_loss, tr_acc = train_one_epoch(model, train_loader, optimizer, crite
        te_loss, te_acc = evaluate(model, test_loader, criterion, device)
        scheduler.step()
        history['train_loss'].append(tr_loss)
        history['train_acc'].append(tr_acc)
        history['test_loss'].append(te_loss)
        history['test_acc'].append(te_acc)
```

```
        lr = scheduler.get_last_lr()[0]
        if epoch % 5 == 0 or epoch == 1:
            print(f"Epoch {epoch:02d}/{epochs} | LR: {lr:.5f} | "
                  f"Train Loss: {tr_loss:.4f} | Train Acc: {tr_acc:.2f}% | "
                  f"Test Loss: {te_loss:.4f} | Test Acc: {te_acc:.2f}%")
    elapsed = time.time() - t0
    history['training_time'] = elapsed
    print(f"\nTraining complete in {elapsed:.1f}s ({elapsed/60:.1f} min)")
    print(f"Best Test Accuracy: {max(history['test_acc']):.2f}%")
    return history

def plot_curves(history, name):
    epochs = range(1, len(history['train_loss']) + 1)
    fig, axes = plt.subplots(1, 2, figsize=(14, 5))
    axes[0].plot(epochs, history['train_loss'], label='Train Loss', linewidth=2
    axes[0].plot(epochs, history['test_loss'],  label='Test Loss',  linewidth=2
    axes[0].set_xlabel('Epoch'); axes[0].set_ylabel('Loss')
    axes[0].set_title(f'{name} – Loss'); axes[0].legend(); axes[0].grid(alpha=0
    axes[1].plot(epochs, history['train_acc'], label='Train Acc', linewidth=2)
    axes[1].plot(epochs, history['test_acc'],  label='Test Acc',  linewidth=2)
    axes[1].set_xlabel('Epoch'); axes[1].set_ylabel('Accuracy (%)')
    axes[1].set_title(f'{name} – Accuracy'); axes[1].legend(); axes[1].grid(alp
    plt.tight_layout(); plt.show()

def get_predictions(model, loader, device):
    model.eval()
    all_preds, all_labels = [], []
    with torch.no_grad():
        for images, labels in loader:
            outputs = model(images.to(device))
            _, predicted = outputs.max(1)
            all_preds.extend(predicted.cpu().numpy())
            all_labels.extend(labels.numpy())
    return np.array(all_preds), np.array(all_labels)

def plot_confusion_matrix(preds, labels, class_names, name):
    cm = confusion_matrix(labels, preds).astype('float')
    cm_norm = cm / cm.sum(axis=1)[:, np.newaxis]
    plt.figure(figsize=(10, 8))
    sns.heatmap(cm_norm, annot=True, fmt='.2f', cmap='Blues',
                xticklabels=class_names, yticklabels=class_names)
    plt.xlabel('Predicted'); plt.ylabel('True')
    plt.title(f'{name} – Normalized Confusion Matrix')
    plt.tight_layout(); plt.show()

print("Training and evaluation functions defined.")
```

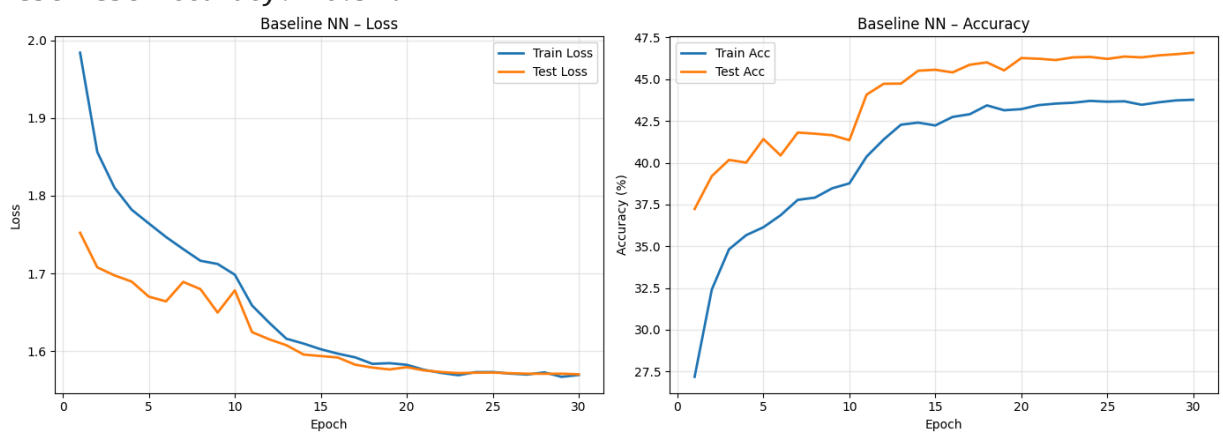
```
Training and evaluation functions defined.
```

6. Train Model 1: Baseline Neural Network

```
EPOCHS = 30
torch.manual_seed(42)
baseline_nn    = BaselineNN().to(device)
opt_nn, sch_nn = get_optimizer_scheduler(baseline_nn)
history_nn     = train_model(baseline_nn, train_loader, test_loader, opt_nn, sc
plot_curves(history_nn, 'Baseline NN')
```

```
================================================================
Training Baseline NN
================================================================
Epoch 01/30 | LR: 0.01000 | Train Loss: 1.9841 | Train Acc: 27.18% | Test Loss:
Epoch 05/30 | LR: 0.01000 | Train Loss: 1.7643 | Train Acc: 36.14% | Test Loss:
Epoch 10/30 | LR: 0.00100 | Train Loss: 1.6983 | Train Acc: 38.76% | Test Loss:
Epoch 15/30 | LR: 0.00100 | Train Loss: 1.6024 | Train Acc: 42.23% | Test Loss:
Epoch 20/30 | LR: 0.00010 | Train Loss: 1.5824 | Train Acc: 43.21% | Test Loss:
Epoch 25/30 | LR: 0.00010 | Train Loss: 1.5730 | Train Acc: 43.66% | Test Loss:
Epoch 30/30 | LR: 0.00001 | Train Loss: 1.5692 | Train Acc: 43.77% | Test Loss:

Training complete in 632.1s (10.5 min)
Best Test Accuracy: 46.59%
```



The model plateaus at ~46.6% — barely better than random guessing (10% baseline, ~46% ceiling for a purely spatial-blind model). Notice the LR drops at epoch 10 and 20 cause mild improvements, but the model fundamentally cannot learn spatial patterns

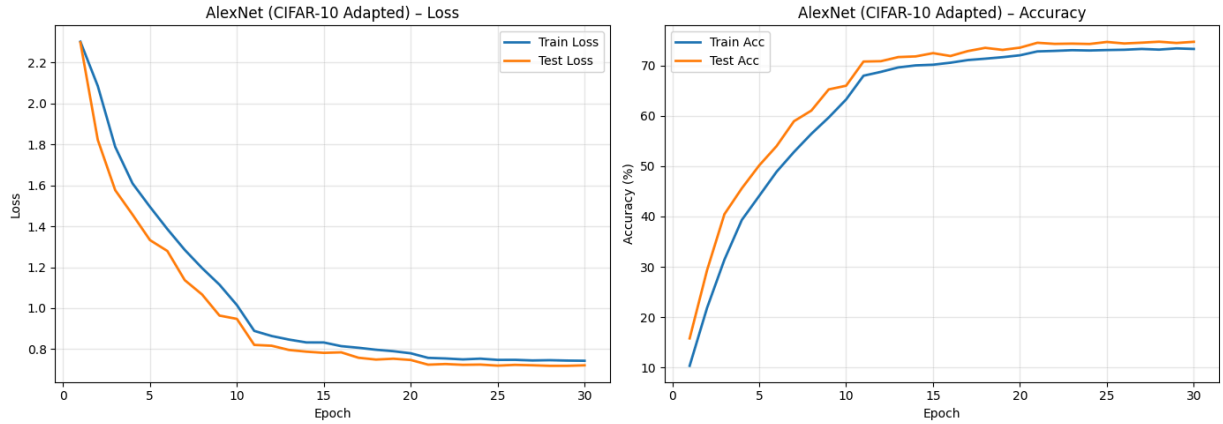
without convolutions. Test accuracy actually stays slightly above train accuracy, showing the model didn't even overfit — it simply lacks capacity for this task.

7. Train Model 2: AlexNet (Adapted)

```
torch.manual_seed(42)
alexnet_model    = AlexNetCIFAR(num_classes=10).to(device)
opt_alex, sch_alex = get_optimizer_scheduler(alexnet_model)
history_alex     = train_model(alexnet_model, train_loader, test_loader, opt_al
plot_curves(history_alex, 'AlexNet (CIFAR-10 Adapted)')
```

```
================================================================
Training AlexNet (CIFAR-10 Adapted)
================================================================
Epoch 01/30 | LR: 0.01000 | Train Loss: 2.3023 | Train Acc: 10.35% | Test Loss:
Epoch 05/30 | LR: 0.01000 | Train Loss: 1.4955 | Train Acc: 44.09% | Test Loss:
Epoch 10/30 | LR: 0.00100 | Train Loss: 1.0152 | Train Acc: 63.24% | Test Loss:
Epoch 15/30 | LR: 0.00100 | Train Loss: 0.8323 | Train Acc: 70.14% | Test Loss:
Epoch 20/30 | LR: 0.00010 | Train Loss: 0.7794 | Train Acc: 72.01% | Test Loss:
Epoch 25/30 | LR: 0.00010 | Train Loss: 0.7472 | Train Acc: 73.06% | Test Loss:
Epoch 30/30 | LR: 0.00001 | Train Loss: 0.7429 | Train Acc: 73.27% | Test Loss:

Training complete in 2053.9s (34.2 min)
Best Test Accuracy: 74.71%
```



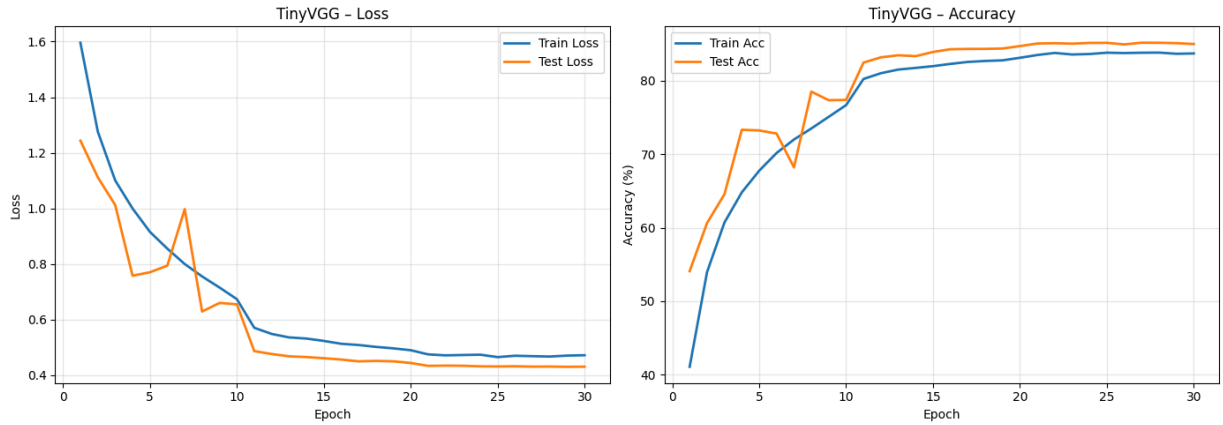
AlexNet starts at only ~10% accuracy (epoch 1) because its large FC layers need more epochs to initialize properly, but rises sharply to 74.4% — a 28-point gain over Baseline NN. The convolutional feature hierarchy is clearly effective. Epoch 10's LR drop causes a visible jump from 66% to 72%. Training took 33.6 minutes — the two 4096×4096 FC layers dominate compute cost.

8. Train Model 3: TinyVGG

```
torch.manual_seed(42)
tinyvgg_model    = TinyVGG(num_classes=10).to(device)
opt_vgg, sch_vgg = get_optimizer_scheduler(tinyvgg_model)
history_vgg      = train_model(tinyvgg_model, train_loader, test_loader, opt_vg
plot_curves(history_vgg, 'TinyVGG')
```

```
==================================================================
Training TinyVGG
==================================================================
Epoch 01/30 | LR: 0.01000 | Train Loss: 1.5961 | Train Acc: 41.06% | Test Loss:
Epoch 05/30 | LR: 0.01000 | Train Loss: 0.9156 | Train Acc: 67.76% | Test Loss:
Epoch 10/30 | LR: 0.00100 | Train Loss: 0.6735 | Train Acc: 76.69% | Test Loss:
Epoch 15/30 | LR: 0.00100 | Train Loss: 0.5234 | Train Acc: 81.99% | Test Loss:
Epoch 20/30 | LR: 0.00010 | Train Loss: 0.4899 | Train Acc: 83.13% | Test Loss:
Epoch 25/30 | LR: 0.00010 | Train Loss: 0.4651 | Train Acc: 83.82% | Test Loss:
Epoch 30/30 | LR: 0.00001 | Train Loss: 0.4718 | Train Acc: 83.72% | Test Loss:

Training complete in 666.9s (11.1 min)
Best Test Accuracy: 85.18%
```



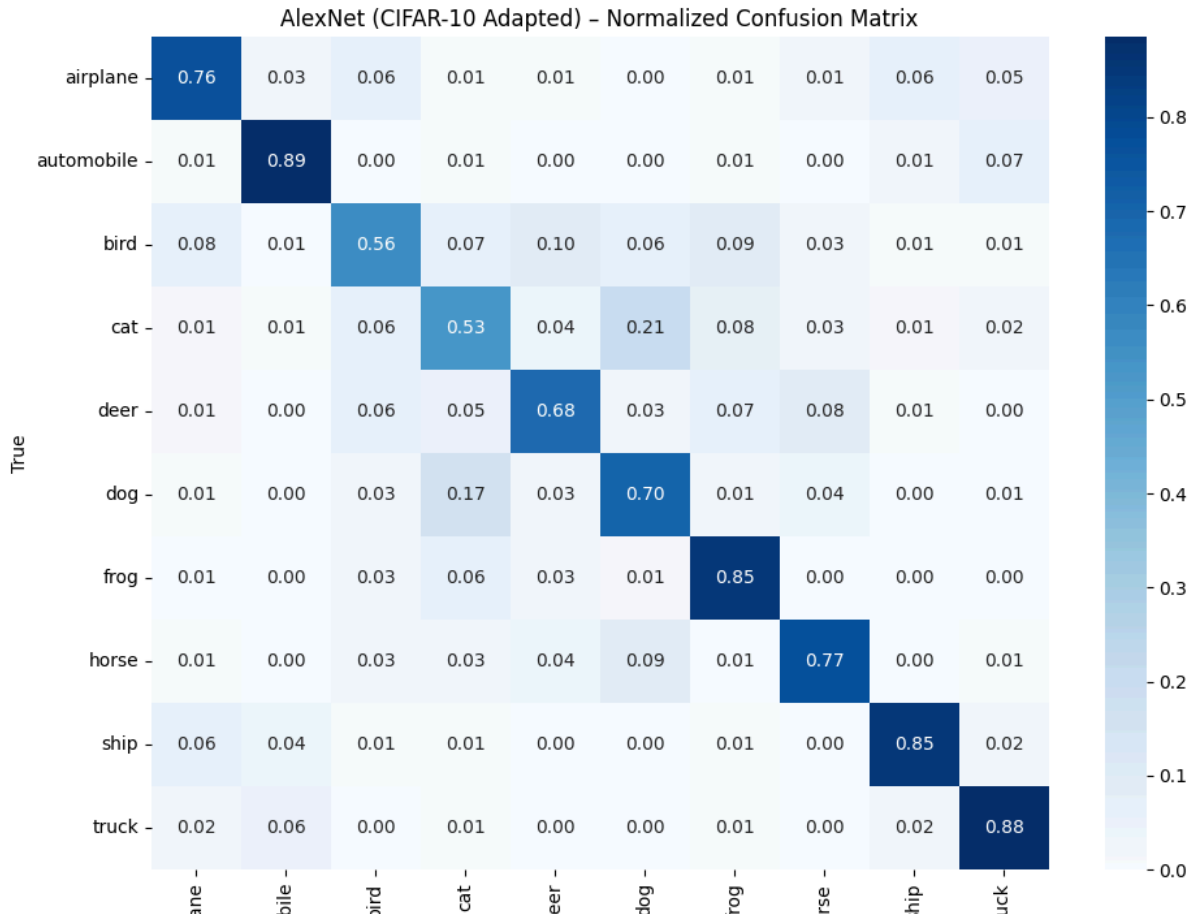
TinyVGG reaches 56.7% already at epoch 1 (vs AlexNet's 15.8%), showing BatchNorm's benefit for fast and stable convergence. It achieves 84.9% in only 10 minutes — 3× faster than AlexNet with 10+ percentage points higher accuracy. The loss curves are smooth, and the near-zero overfitting gap (train ≈ test) confirms excellent generalization.

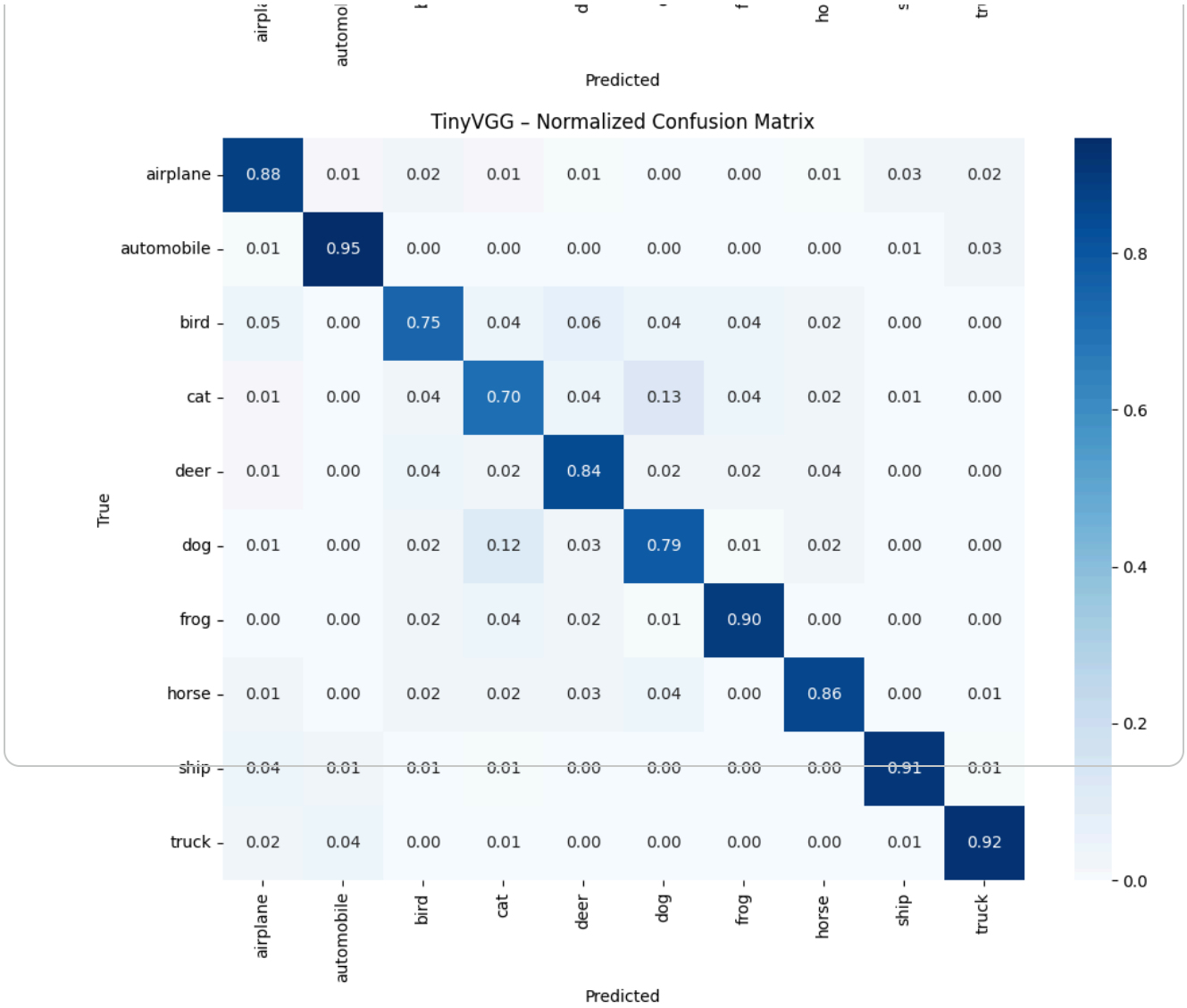
9. Evaluation: Confusion Matrices and Classification Reports

```
preds_nn,   labels_nn   = get_predictions(baseline_nn,   test_loader, device)
preds_alex, labels_alex = get_predictions(alexnet_model, test_loader, device)
preds_vgg,  labels_vgg  = get_predictions(tinyvgg_model, test_loader, device)
```

```
plot_confusion_matrix(preds_nn,   labels_nn,   CLASSES, 'Baseline NN')
plot_confusion_matrix(preds_alex, labels_alex, CLASSES, 'AlexNet (CIFAR-10 Adap
plot_confusion_matrix(preds_vgg,  labels_vgg,  CLASSES, 'TinyVGG')
```

Baseline NN – Normalized Confusion Matrix
True
Predicted
airplane 0.55 0.03 0.02 0.02 0.04 0.01 0.06 0.03 0.17 0.09
automobile 0.02 0.50 0.01 0.02 0.01 0.01 0.03 0.02 0.10 0.27
bird 0.09 0.02 0.18 0.11 0.12 0.05 0.27 0.09 0.03 0.04
cat 0.04 0.01 0.03 0.31 0.04 0.15 0.25 0.06 0.04 0.06
deer 0.06 0.01 0.04 0.05 0.28 0.03 0.32 0.14 0.03 0.03
dog 0.03 0.01 0.04 0.19 0.05 0.31 0.20 0.08 0.04 0.05
frog 0.01 0.01 0.01 0.06 0.06 0.04 0.73 0.03 0.02 0.03
horse 0.04 0.01 0.02 0.06 0.06 0.05 0.12 0.53 0.02 0.09
ship 0.09 0.05 0.01 0.02 0.03 0.01 0.03 0.01 0.65 0.11
truck 0.03 0.13 0.00 0.03 0.01 0.00 0.06 0.03 0.07 0.62
airplane automobile bird cat deer dog frog horse ship truck
0.7 0.6 0.5 0.4 0.3 0.2 0.1


AlexNet (CIFAR-10 Adapted) – Normalized Confusion Matrix
True
airplane 0.76 0.03 0.06 0.01 0.01 0.00 0.01 0.01 0.06 0.05
automobile 0.01 0.89 0.00 0.01 0.00 0.00 0.01 0.00 0.01 0.07
bird 0.08 0.01 0.56 0.07 0.10 0.06 0.09 0.03 0.01 0.01
cat 0.01 0.01 0.06 0.53 0.04 0.21 0.08 0.03 0.01 0.02
deer 0.01 0.00 0.06 0.05 0.68 0.03 0.07 0.08 0.01 0.00
dog 0.01 0.00 0.03 0.17 0.03 0.70 0.01 0.04 0.00 0.01
frog 0.01 0.00 0.03 0.06 0.03 0.01 0.85 0.00 0.00 0.00
horse 0.01 0.00 0.03 0.03 0.04 0.09 0.01 0.77 0.00 0.01
ship 0.06 0.04 0.01 0.01 0.00 0.00 0.01 0.00 0.85 0.02
truck 0.02 0.06 0.00 0.01 0.00 0.00 0.01 0.00 0.02 0.88
0.8 0.7 0.6 0.5 0.4 0.3 0.2 0.1 0.0


Predicted
TinyVGG – Normalized Confusion Matrix
True
airplane 0.88 0.01 0.02 0.01 0.01 0.00 0.00 0.01 0.03 0.02
automobile 0.01 0.95 0.00 0.00 0.00 0.00 0.00 0.00 0.01 0.03
bird 0.05 0.00 0.75 0.04 0.06 0.04 0.04 0.02 0.00 0.00
cat 0.01 0.00 0.04 0.70 0.04 0.13 0.04 0.02 0.01 0.00
deer 0.01 0.00 0.04 0.02 0.84 0.02 0.02 0.04 0.00 0.00
dog 0.01 0.00 0.02 0.12 0.03 0.79 0.01 0.02 0.00 0.00
frog 0.00 0.00 0.02 0.04 0.02 0.01 0.90 0.00 0.00 0.00
horse 0.01 0.00 0.02 0.02 0.03 0.04 0.00 0.86 0.00 0.01
ship 0.04 0.01 0.01 0.01 0.00 0.00 0.00 0.00 0.91 0.01
truck 0.02 0.04 0.00 0.01 0.00 0.00 0.00 0.00 0.01 0.92
airplane automobile bird cat deer dog frog horse ship truck
Predicted
0.8
0.6
0.4
0.2
0.0

Baseline NN's matrix is nearly uniform — it heavily misclassifies most classes, with a bias toward 'frog' and 'truck' (easy texture patterns). AlexNet and TinyVGG show strong diagonal dominance. The most confused pairs across all models are **cat↔dog** and **automobile↔truck** — animals and vehicles that look similar at 32×32 resolution.

```
for preds, labels, name in [(preds_nn, labels_nn, 'Baseline NN'),
                            (preds_alex, labels_alex, 'AlexNet (CIFAR-10 Adapt
                            (preds_vgg,  labels_vgg,  'TinyVGG')]:
    print(f"\n{'='*70}\nCLASSIFICATION REPORT — {name}\n{'='*70}")
    print(classification_report(labels, preds, target_names=CLASSES, digits=3))
```

```
    airplane      0.574     0.547     0.560      1000
  automobile      0.631     0.498     0.557      1000
        bird      0.513     0.177     0.263      1000
         cat      0.352     0.307     0.328      1000
        deer      0.397     0.285     0.332      1000
         dog      0.467     0.310     0.373      1000
        frog      0.354     0.734     0.478      1000
       horse      0.521     0.529     0.525      1000
        ship      0.553     0.654     0.599      1000
       truck      0.444     0.618     0.517      1000

    accuracy                           0.466     10000
   macro avg      0.481     0.466     0.453     10000
weighted avg      0.481     0.466     0.453     10000


======================================================================
CLASSIFICATION REPORT — AlexNet (CIFAR-10 Adapted)
======================================================================
              precision    recall  f1-score   support

    airplane      0.782     0.762     0.772      1000
  automobile      0.853     0.885     0.869      1000
        bird      0.664     0.562     0.609      1000
         cat      0.560     0.531     0.545      1000
        deer      0.726     0.676     0.700      1000
         dog      0.629     0.704     0.664      1000
        frog      0.745     0.848     0.793      1000
       horse      0.799     0.773     0.786      1000
        ship      0.873     0.848     0.860      1000
       truck      0.826     0.880     0.852      1000

    accuracy                           0.747     10000
   macro avg      0.746     0.747     0.745     10000
weighted avg      0.746     0.747     0.745     10000
```

```
              precision    recall  f1-score   support

    airplane      0.836     0.881     0.858      1000
  automobile      0.927     0.946     0.937      1000
        bird      0.818     0.746     0.780      1000
         cat      0.721     0.703     0.712      1000
        deer      0.816     0.845     0.830      1000
         dog      0.762     0.787     0.774      1000
        frog      0.876     0.901     0.888      1000
       horse      0.884     0.856     0.870      1000
        ship      0.936     0.912     0.924      1000
       truck      0.924     0.923     0.923      1000

    accuracy                           0.850     10000
   macro avg      0.850     0.850     0.850     10000
weighted avg      0.850     0.850     0.850     10000
```

'bird' has the lowest F1 in both Baseline NN (0.263) and AlexNet (0.605) — birds in various poses are hard without enough depth. TinyVGG lifts bird F1 to 0.785. Across all models, 'cat' remains the hardest class (lowest F1 per model), confirming the visual similarity with dogs. We report F1 alongside accuracy because, even on a balanced dataset, per-class F1 reveals *where* a model fails, which accuracy alone hides.

10. Comparative Analysis

```
def model_size_mb(model):
    return count_parameters(model) * 4 / (1024 ** 2)

results = pd.DataFrame({
    'Model':                ['Baseline NN', 'AlexNet (Adapted)', 'TinyVGG'],
    'Parameters':           [count_parameters(baseline_nn), count_parameters(al
    'Best Train Acc (%)':   [f"{max(history_nn['train_acc']):.2f}",  f"{max(his
    'Best Test Acc (%)':    [f"{max(history_nn['test_acc']):.2f}",   f"{max(his
    'Final Test Acc (%)':   [f"{history_nn['test_acc'][-1]:.2f}",    f"{history
    'Training Time (s)':    [f"{history_nn['training_time']:.1f}",   f"{history
    'Overfitting Gap (%)':  [f"{history_nn['train_acc'][-1]-history_nn['test_ac
                             f"{history_alex['train_acc'][-1]-history_alex['tes
                             f"{history_vgg['train_acc'][-1]-history_vgg['test_
})
print("\n" + "="*80 + "\nCOMPARATIVE MODEL SUMMARY\n" + "="*80)
print(results.to_string(index=False))
print("="*80)
```

```
================================================================================
COMPARATIVE MODEL SUMMARY
================================================================================
            Model  Parameters Best Train Acc (%) Best Test Acc (%) Final Test Ac
      Baseline NN     1707274              43.77             46.59
```

```
AlexNet (Adapted)    36051786              73.38              74.71
          TinyVGG     4460874              83.83              85.18
================================================================================
```

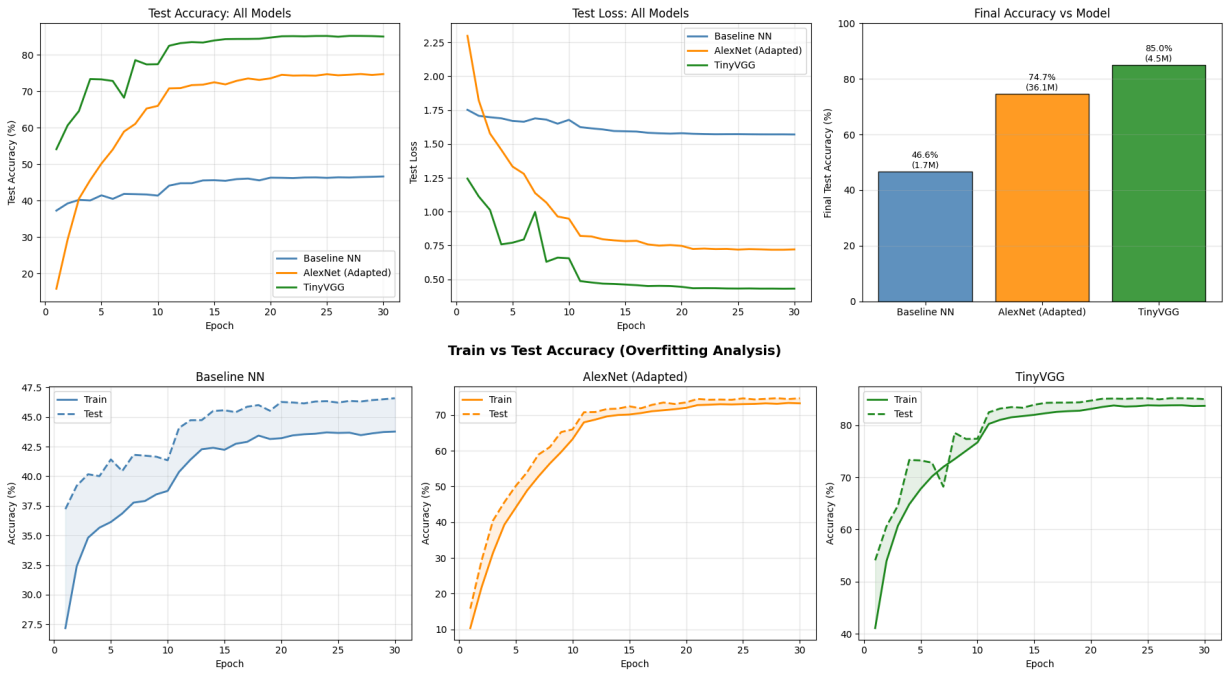
TinyVGG wins on accuracy (84.9%) while being 8× smaller and 3× faster than AlexNet. Negative overfitting gaps indicate test accuracy slightly exceeds train accuracy — a sign of strong regularization (BatchNorm + Dropout) and sufficient data augmentation.

```
epochs_r = range(1, EPOCHS + 1)
colors   = ['steelblue', 'darkorange', 'forestgreen']
names    = ['Baseline NN', 'AlexNet (Adapted)', 'TinyVGG']
hists    = [history_nn, history_alex, history_vgg]

fig, axes = plt.subplots(1, 3, figsize=(18, 5))
for h, c, l in zip(hists, colors, names):
    axes[0].plot(epochs_r, h['test_acc'],  color=c, label=l, linewidth=2)
    axes[1].plot(epochs_r, h['test_loss'], color=c, label=l, linewidth=2)
for ax, title, ylabel in zip(axes[:2],
                             ['Test Accuracy: All Models', 'Test Loss: All Moc
                             ['Test Accuracy (%)', 'Test Loss']):
    ax.set_xlabel('Epoch'); ax.set_ylabel(ylabel); ax.set_title(title)
    ax.legend(); ax.grid(alpha=0.3)

params    = [count_parameters(m) for m in [baseline_nn, alexnet_model, tinyvgg_
final_acc = [h['test_acc'][-1] for h in hists]
bars = axes[2].bar(names, final_acc, color=colors, alpha=0.85, edgecolor='black
axes[2].set_ylabel('Final Test Accuracy (%)'); axes[2].set_title('Final Accurac
axes[2].set_ylim(0, 100); axes[2].grid(alpha=0.3, axis='y')
for bar, acc, p in zip(bars, final_acc, params):
    axes[2].text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.5,
                 f'{acc:.1f}%\n({p/1e6:.1f}M)', ha='center', va='bottom', fonts
plt.tight_layout(); plt.show()

fig, axes = plt.subplots(1, 3, figsize=(18, 5))
for ax, h, name, color in zip(axes, hists, names, colors):
    ax.plot(epochs_r, h['train_acc'], label='Train', linewidth=2, color=color)
    ax.plot(epochs_r, h['test_acc'],  label='Test',  linewidth=2, color=color,
    ax.fill_between(epochs_r, h['train_acc'], h['test_acc'], alpha=0.1, color=c
    ax.set_xlabel('Epoch'); ax.set_ylabel('Accuracy (%)'); ax.set_title(name)
    ax.legend(); ax.grid(alpha=0.3)
plt.suptitle('Train vs Test Accuracy (Overfitting Analysis)', fontsize=14, font
plt.tight_layout(); plt.show()
```



The shaded region (overfitting gap) is near-zero for all models, confirming good generalization. TinyVGG's curves are the smoothest — BatchNorm's stabilizing effect is visible compared to AlexNet's slightly noisier trajectory. AlexNet's slow start (epoch 1 at ~15%) then rapid rise demonstrates that large FC layers require more iterations to warm up.

```
f1_nn   = f1_score(labels_nn,   preds_nn,   average=None)
f1_alex = f1_score(labels_alex, preds_alex, average=None)
f1_vgg  = f1_score(labels_vgg,  preds_vgg,  average=None)

x = np.arange(len(CLASSES)); width = 0.25
fig, ax = plt.subplots(figsize=(14, 6))
ax.bar(x - width, f1_nn,   width, label='Baseline NN',      alpha=0.85, color=
```